

Python Coding Interview Cheatsheet

Patterns • Priorities • Popularity • Code Templates • LeetCode Mappings

Priority & Popularity Overview

#	Pattern	Priority	Popularity	Representative LeetCode
1	Two Pointers: One Input, Opposite Ends	High	★★★★★	125, 680, 11
2	Two Pointers: Two Inputs, Exhaust Both	High	★★★★★	21, 88, 392
3	Sliding Window	High	★★★★★	3, 76, 209
4	Build a Prefix Sum	High	★★★★★	560, 303, 304
5	Efficient String Building	Medium	★★★★■	443, 271, 68
6	Linked List: Fast and Slow Pointer	High	★★★★★	141, 142, 876
7	Reversing a Linked List	High	★★★★★	206, 92, 25
8	Find # of Subarrays Meeting Exact Criteria	High	★★★★★	560, 930, 1248
9	Monotonic Increasing Stack	High	★★★★★	739, 496, 84
10	Binary Tree: DFS (Recursive)	High	★★★★■	104, 112, 437
11	Binary Tree: DFS (Iterative)	High	★★★★■	144, 94, 145
12	Binary Tree: BFS (Level Order)	High	★★★★★	102, 103, 107
13	Graph: DFS (Recursive)	High	★★★★★	200, 695, 547
14	Graph: DFS (Iterative)	Medium	★★★★■	133, 417
15	Graph: BFS	High	★★★★★	127, 133, 994
16	Find Top K Elements with Heap	High	★★★★★	347, 215, 973
17	Binary Search (Classic)	High	★★★★★	704, 35, 278
18	Binary Search: Left-most Insertion Point	High	★★★★■	34, 852
19	Binary Search: Right-most Insertion Point	High	★★★★■	34
20	Binary Search for Greedy (Parametric)	High	★★★★★	410, 1011, 875
21	Backtracking	High	★★★★★	46, 78, 39
22	Dynamic Programming: Top-down Memoization	High	★★★★★	70, 198, 322
23	Build a Trie	Medium	★★★★■	208, 211, 212
24	Dijkstra's Algorithm	Advanced	★★★██	743, 1631
25	Prim's Algorithm (MST)	Advanced	★★★██	1135, 1584
26	Kruskal's Algorithm (MST)	Advanced	★★★██	1584, 1135

1. Two Pointers: One Input, Opposite Ends

Priority: High **Popularity:** ★★★★★

Intent: Check symmetry or converge from ends on a single sequence.

Code Template

```
def is_palindrome(word: str) -> bool:
    left, right = 0, len(word) - 1
    while left < right:
        if word[left] != word[right]:
```

```

        return False
    left += 1
    right -= 1
    return True

```

LeetCode References

ID	Title
125	Valid Palindrome
680	Valid Palindrome II
11	Container With Most Water

Highlights / Pitfalls

- $O(n)$ time, $O(1)$ space.
- Great for palindromes, two-sum on sorted arrays, water container.
- Beware off-by-one; use while left < right.

2. Two Pointers: Two Inputs, Exhaust Both

Priority: High Popularity: ★★★★★

Intent: Merge or compare two sorted sequences in linear time.

Code Template

```

def merge_sorted_arrays(arr_a, arr_b):
    i, j, merged = 0, 0, []
    while i < len(arr_a) and j < len(arr_b):
        if arr_a[i] <= arr_b[j]:
            merged.append(arr_a[i]); i += 1
        else:
            merged.append(arr_b[j]); j += 1
    merged.extend(arr_a[i:]); merged.extend(arr_b[j:])
    return merged

```

LeetCode References

ID	Title
21	Merge Two Sorted Lists
88	Merge Sorted Array
392	Is Subsequence

Highlights / Pitfalls

- Stable merge in $O(n+m)$.
- Use when both inputs are sorted.
- Boundary safety: extend the remainder after loop.

3. Sliding Window

Priority: High Popularity: ★★★★★

Intent: Maintain a window over a sequence to satisfy a constraint and slide it.

Code Template

```

def longest_substring_no_repeat(text: str) -> int:
    last_seen, left, best = {}, 0, 0
    for right, ch in enumerate(text):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1
        last_seen[ch] = right
        best = max(best, right - left + 1)
    return best

```

LeetCode References

ID	Title
3	Longest Substring Without Repeating Characters
76	Minimum Window Substring
209	Minimum Size Subarray Sum

Highlights / Pitfalls

- Two flavors: fixed-size vs variable-size windows.
- Use dict/counter to track counts or positions.
- Move left pointer only when constraint breaks.

4. Build a Prefix Sum

Priority: High Popularity: ★★★★★

Intent: Precompute cumulative sums for O(1) range queries or subarray math.

Code Template

```
def build_prefix_sum(nums):
    prefix = [0]
    for value in nums:
        prefix.append(prefix[-1] + value)
    return prefix # sum(i..j) = prefix[j+1] - prefix[i]
```

LeetCode References

ID	Title
560	Subarray Sum Equals K
303	Range Sum Query - Immutable
304	Range Sum Query 2D - Immutable

Highlights / Pitfalls

- Combine with hashmap of prefix frequencies for counting subarrays.
- 2D variant uses inclusion–exclusion.

5. Efficient String Building

Priority: Medium Popularity: ★★★★■

Intent: Build strings in O(n) using join, avoid quadratic concatenation.

Code Template

```
def build_string(words):
    return "".join(words) # Avoid += in loops
```

LeetCode References

ID	Title
443	String Compression
271	Encode and Decode Strings (premium-like)
68	Text Justification

Highlights / Pitfalls

- `"".join(iterable)` is linear; repeated concatenation is quadratic.
- Use list buffer then join.

6. Linked List: Fast and Slow Pointer

Priority: High **Popularity:** ★★★★★

Intent: Detect cycles or find the middle using two speeds.

Code Template

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            return True
    return False
```

LeetCode References

ID	Title
141	Linked List Cycle
142	Linked List Cycle II
876	Middle of the Linked List

Highlights / Pitfalls

- Floyd's cycle detection: meet means cycle.
- To find cycle start: reset one pointer to head and advance both 1 step.

7. Reversing a Linked List

Priority: High **Popularity:** ★★★★★

Intent: Iteratively reverse pointers to invert the list.

Code Template

```
def reverse_list(head):
    prev, curr = None, head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev, curr = curr, nxt
    return prev
```

LeetCode References

ID	Title
206	Reverse Linked List
92	Reverse Linked List II
25	Reverse Nodes in k-Group

Highlights / Pitfalls

- Core routine for many linked list problems.
- Be careful to store next before rewiring.

8. Find # of Subarrays Meeting Exact Criteria

Priority: High **Popularity:** ★★★★★

Intent: Count subarrays with exact sum or property using prefix + hashmap.

Code Template

```
def count_subarrays_sum_k(nums, target_sum):
    counts = {0: 1}
    prefix, total = 0, 0
```

```

for value in nums:
    prefix += value
    total += counts.get(prefix - target_sum, 0)
    counts[prefix] = counts.get(prefix, 0) + 1
return total

```

LeetCode References

ID	Title
560	Subarray Sum Equals K
930	Binary Subarrays With Sum
1248	Count Number of Nice Subarrays

Highlights / Pitfalls

- Time $O(n)$, space $O(n)$.
- Initialize counts[0]=1 to count prefixes equal to target.

9. Monotonic Increasing Stack

Priority: High Popularity: ★★★★★

Intent: Maintain indices in increasing order to solve 'next greater/smaller' efficiently.

Code Template

```

def next_greater_elements(values):
    stack, answer = [], [-1] * len(values) # stack holds indices
    for i, val in enumerate(values):
        while stack and values[stack[-1]] < val:
            answer[stack.pop()] = val
        stack.append(i)
    return answer

```

LeetCode References

ID	Title
739	Daily Temperatures
496	Next Greater Element I
84	Largest Rectangle in Histogram

Highlights / Pitfalls

- Common for span/stock/histogram problems.
- Store indices to compute distances/areas.

10. Binary Tree: DFS (Recursive)

Priority: High Popularity: ★★★★★

Intent: Depth-first traversal via recursion (pre/in/post).

Code Template

```

def preorder(node):
    if not node: return []
    return [node.val] + preorder(node.left) + preorder(node.right)

```

LeetCode References

ID	Title
104	Maximum Depth of Binary Tree
112	Path Sum

437	Path Sum III
-----	--------------

Highlights / Pitfalls

- Be mindful of recursion depth $\sim 1e4$ in Python (may need iterative).

11. Binary Tree: DFS (Iterative)

Priority: High Popularity: ★★★★★

Intent: Use explicit stack to avoid recursion limits.

Code Template

```
def preorder_iterative(root):
    stack, out = [root], []
    while stack:
        node = stack.pop()
        if node:
            out.append(node.val)
            stack.append(node.right)
            stack.append(node.left)
    return out
```

LeetCode References

ID	Title
144	Binary Tree Preorder Traversal
94	Binary Tree Inorder Traversal
145	Binary Tree Postorder Traversal

Highlights / Pitfalls

- Push right first so left is processed first.

12. Binary Tree: BFS (Level Order)

Priority: High Popularity: ★★★★★

Intent: Breadth-first traversal across levels with a queue.

Code Template

```
from collections import deque

def level_order(root):
    if not root: return []
    result, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        result.append(level)
    return result
```

LeetCode References

ID	Title
102	Binary Tree Level Order Traversal
103	Binary Tree Zigzag Level Order
107	Binary Tree Level Order Traversal II

Highlights / Pitfalls

- Use fixed loop on current queue length to split levels.

13. Graph: DFS (Recursive)

Priority: High **Popularity:** ★★★★★

Intent: Explore connected components or paths recursively.

Code Template

```
def dfs_graph(node, graph, visited):
    if node in visited: return
    visited.add(node)
    for nei in graph[node]:
        dfs_graph(nei, graph, visited)
```

LeetCode References

ID	Title
200	Number of Islands
695	Max Area of Island
547	Number of Provinces

Highlights / Pitfalls

- Works for grids (convert to graph or inline neighbors).

14. Graph: DFS (Iterative)

Priority: Medium **Popularity:** ★★★★★■

Intent: Non-recursive graph exploration using a stack.

Code Template

```
def dfs_graph_iterative(start, graph):
    stack, visited = [start], set()
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            stack.extend(graph[node])
    return visited
```

LeetCode References

ID	Title
133	Clone Graph
417	Pacific Atlantic Water Flow

Highlights / Pitfalls

- Safer than recursion for deep graphs.

15. Graph: BFS

Priority: High **Popularity:** ★★★★★

Intent: Shortest path in unweighted graphs / layered expansion.

Code Template

```
from collections import deque

def bfs_graph(start, graph):
    q, visited = deque([start]), {start}
    while q:
```

```

        node = q.popleft()
        for nei in graph[node]:
            if nei not in visited:
                visited.add(nei)
                q.append(nei)
    return visited

```

LeetCode References

ID	Title
127	Word Ladder
133	Clone Graph
994	Rotting Oranges

Highlights / Pitfalls

- Add distance tracking by enqueueing (node, dist).

16. Find Top K Elements with Heap

Priority: High **Popularity:** ★★★★★

Intent: Use min-heap of size k or `heapq.nlargest` for simplicity.

Code Template

```

import heapq
from collections import Counter

def top_k_frequent(nums, k):
    freq = Counter(nums)
    return [num for num, _ in heapq.nlargest(k, freq.items(), key=lambda it: it[1])]

```

LeetCode References

ID	Title
347	Top K Frequent Elements
215	Kth Largest Element in an Array
973	K Closest Points to Origin

Highlights / Pitfalls

- For streaming, maintain a size-k min-heap.
- `heapq` is a min-heap; invert keys or use `nlargest`.

17. Binary Search (Classic)

Priority: High **Popularity:** ★★★★★

Intent: Search in sorted or answer-space binary search.

Code Template

```

def binary_search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target: return mid
        if nums[mid] < target: left = mid + 1
        else: right = mid - 1
    return -1

```

LeetCode References

ID	Title
----	-------

704	Binary Search
35	Search Insert Position
278	First Bad Version

Highlights / Pitfalls

- Use left <= right; update boundaries carefully.

18. Binary Search: Left-most Insertion Point

Priority: High Popularity: ★★★★★

Intent: Find first index where target can be inserted to keep order.

Code Template

```
import bisect

def leftmost_index(nums, target):
    return bisect.bisect_left(nums, target)
```

LeetCode References

ID	Title
34	Find First and Last Position of Element in Sorted Array
852	Peak Index in a Mountain Array

Highlights / Pitfalls

- Left boundary for duplicates; may equal len(nums).

19. Binary Search: Right-most Insertion Point

Priority: High Popularity: ★★★★★

Intent: Find last index where target could be inserted on the right.

Code Template

```
import bisect

def rightmost_index(nums, target):
    return bisect.bisect_right(nums, target) - 1
```

LeetCode References

ID	Title
34	Find First and Last Position of Element in Sorted Array

Highlights / Pitfalls

- Right boundary for duplicates; may be -1 if all < target.

20. Binary Search for Greedy (Parametric)

Priority: High Popularity: ★★★★★

Intent: Search the minimum feasible/maximum achievable parameter.

Code Template

```
def min_capacity(weights, days):
    left, right = max(weights), sum(weights)
    while left < right:
        mid = (left + right) // 2
        need, curr = 1, 0
        for w in weights:
```

```

        if curr + w > mid:
            need += 1
            curr = 0
        curr += w
    if need > days:
        left = mid + 1
    else:
        right = mid
return left

```

LeetCode References

ID	Title
410	Split Array Largest Sum
1011	Capacity To Ship Packages Within D Days
875	Koko Eating Bananas

Highlights / Pitfalls

- Feasibility check in $O(n)$; binary search over answer range.

21. Backtracking

Priority: High Popularity: ★★★★★

Intent: Try choices, recurse, and undo (choose → explore → unchoose).

Code Template

```

def subsets(nums):
    result = []
    def backtrack(start, path):
        result.append(path[:])
        for i in range(start, len(nums)):
            backtrack(i + 1, path + [nums[i]])
    backtrack(0, [])
    return result

```

LeetCode References

ID	Title
46	Permutations
78	Subsets
39	Combination Sum
22	Generate Parentheses

Highlights / Pitfalls

- Prune with constraints to avoid TLE.
- Use sorted + skip duplicates for unique subsets/permutations.

22. Dynamic Programming: Top-down Memoization

Priority: High Popularity: ★★★★★

Intent: Cache overlapping subproblems via recursion + memo.

Code Template

```

from functools import lru_cache

def min_coins(amount, coins):
    @lru_cache(None)
    def dp(rem):
        if rem == 0: return 0

```

```

        if rem < 0: return float('inf')
        return 1 + min(dp(rem - c) for c in coins)
ans = dp(amount)
return ans if ans != float('inf') else -1

```

LeetCode References

ID	Title
70	Climbing Stairs
198	House Robber
322	Coin Change

Highlights / Pitfalls

- Top-down easier to write; convert to bottom-up for optimization.
- Use lru_cache for automatic memo.

23. Build a Trie

Priority: Medium **Popularity:** ★★★★★

Intent: Prefix tree for fast word/prefix queries.

Code Template

```

class TrieNode:
    def __init__(self):
        self.children, self.is_end = {}, False

class Trie:
    def __init__(self):
        self.root = TrieNode()
    def insert(self, word: str) -> None:
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.is_end = True
    def search(self, word: str) -> bool:
        node = self.root
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.is_end

```

LeetCode References

ID	Title
208	Implement Trie
211	Add and Search Word
212	Word Search II

Highlights / Pitfalls

- Space heavy; good for many short queries.
- Can store counts/indices at nodes for extra features.

24. Dijkstra's Algorithm

Priority: Advanced **Popularity:** ★★★★★

Intent: Shortest paths with non-negative weights using a min-heap.

Code Template

```
import heapq

def dijkstra(graph, start):
    dist = {node: float('inf') for node in graph}
    dist[start] = 0
    pq = [(0, start)]
    while pq:
        d, node = heapq.heappop(pq)
        if d > dist[node]:
            continue
        for nei, w in graph[node]:
            nd = d + w
            if nd < dist[nei]:
                dist[nei] = nd
                heapq.heappush(pq, (nd, nei))
    return dist
```

LeetCode References

ID	Title
743	Network Delay Time
1631	Path With Minimum Effort

Highlights / Pitfalls

- Stop early if target popped with minimal distance.

25. Prim's Algorithm (MST)

Priority: Advanced Popularity: ★★★

Intent: Grow MST by always adding the cheapest edge to the tree.

Code Template

```
import heapq

def prim(graph, start=0):
    seen, total, pq = {start}, 0, []
    for nei, w in graph[start]:
        heapq.heappush(pq, (w, start, nei))
    while pq:
        w, u, v = heapq.heappop(pq)
        if v in seen:
            continue
        seen.add(v); total += w
        for nei, w2 in graph[v]:
            if nei not in seen:
                heapq.heappush(pq, (w2, v, nei))
    return total
```

LeetCode References

ID	Title
1135	Connecting Cities With Minimum Cost
1584	Min Cost to Connect All Points

Highlights / Pitfalls

- Graph must be connected to span all nodes.

26. Kruskal's Algorithm (MST)

Priority: Advanced Popularity: ★★★

Intent: Pick edges in ascending weight; union-find to avoid cycles.

Code Template

```
def kruskal(n, edges):
    parent = list(range(n))
    rank = [0] * n
    def find(x):
        if x != parent[x]:
            parent[x] = find(parent[x])
        return parent[x]
    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb: return False
        if rank[ra] < rank[rb]:
            parent[ra] = rb
        elif rank[ra] > rank[rb]:
            parent[rb] = ra
        else:
            parent[rb] = ra
            rank[ra] += 1
        return True
    total = 0
    for w, u, v in sorted(edges):
        if union(u, v):
            total += w
    return total
```

LeetCode References

ID	Title
1584	Min Cost to Connect All Points
1135	Connecting Cities With Minimum Cost

Highlights / Pitfalls

- Union by rank + path compression for near $O(\alpha(n))$.